

Day-69-Basic Debugging and Error Handling for Data Programs

1. Day Number

Day 69

2. Topic Name

Debugging file and API programs

Today you will learn how to make small data programs behave safely when something goes wrong while reading a file, parsing JSON, or getting data from an API.

3. Connection to Previous Learning

You already learned basic try-except and manual debugging.

Yesterday, you also learned that APIs can return JSON data.

Today, you will combine those skills.

Instead of assuming external data is always available and correct, you will learn to think:

| “What could fail, and how can my program handle that failure without crashing?”

4. Important Topics

The main ideas for today are:

- try
- except
- print debugging
- file errors
- invalid JSON
- request failure

The goal is **not** to handle every possible error.

The goal is to recognize common failures and respond to them clearly.

5. Foundational Notes

What is debugging?

Debugging means finding and fixing problems in a program.

For example, suppose your program should read:

products.json

but Python says the file cannot be found.

You might debug by printing useful information:

```
print("Trying to open the file...")
```

or:

```
print(filename)
```

These prints help you understand where the program reached and what values it was using.

This is often called **print debugging**.

What is error handling?

Error handling means preparing your program for situations that may fail.

For example:

```
try:
    # risky operation
except:
    # handle the problem
```

Think of it like this:

```
TRY to do something.

IF it fails,
handle the problem instead of crashing.
```

For beginner programs, try-except is especially useful when working with external data.

6. Why External Data Can Fail Even When Python Syntax Is Correct

Your Python code can be perfectly valid and still fail.

Why?

Because your program may depend on something **outside your Python code**.

Consider this program idea:

```
Python program
  ↓
read data.json
```

Your Python syntax may be correct, but perhaps:

```
data.json does not exist
```

or:

```
data.json contains broken JSON
```

Similarly, with an API:

```
Python program
  ↓
Internet
  ↓
API server
```

Your program could fail because:

- the internet is unavailable
- the server is unavailable
- the server returns an error
- the response is not valid JSON
- expected data is missing

So there are two different ideas:

```
Correct Python code
```

does **not always mean**

```
Successful program execution
```

External resources can cause problems that your program cannot completely control.

7. Easy Example

Imagine you want to load customer information from:

```
customer.json
```

The data might normally look like:

```
{
  "name": "Riya",
  "city": "Bengaluru"
}
```

A basic program might attempt:

```
print("Loading customer data...")

try:
    # open the file
    # read the JSON data

except:
    print("Could not load customer data.")
```

Notice the important idea.

Instead of the program suddenly stopping with a large error message, it gives the user a simple explanation.

Successful situation

```
Loading customer data...
Customer: Riya
City: Bengaluru
```

Failed situation

```
Loading customer data...
Could not load customer data.
```

For now, focus on the **flow**, not on handling every specific Python exception.

8. Problem Statement

Create a simple **data-loading function**.

Your function should:

1. receive the name of a data file
2. attempt to load the data
3. print or return useful information when loading succeeds
4. handle one possible failure gracefully
5. display a simple message instead of letting the program crash

For example:

```
load_data("customer.json")
```

If the file exists and contains valid data:

Data loaded successfully.

If something goes wrong:

Could not load the data.

For today's exercise, handle **one failure clearly** rather than trying to catch every possible problem.

9. Concepts Used

You will practice:

- functions
- function parameters
- files
- JSON data
- try
- except
- conditional thinking
- print debugging
- graceful failure

You are combining several topics you have already learned.

A possible structure is:

```
function
  ↓
try to load data
  ↓
success?
  ✓       \
yes       no
  ↓       ↓
use data  show helpful message
```

10. Thought Process

Before writing code, think through the program.

Step 1: What data am I trying to load?

For example:

customer.json

Step 2: Which operation might fail?

Opening or reading the external file could fail.

So that operation belongs inside your try section.

Step 3: What should happen when it works?

For example:

```
Data loaded successfully.
```

Then you can use the loaded data.

Step 4: What should happen when it fails?

Instead of crashing:

```
Could not load the data.
```

Step 5: Can print debugging help?

Yes.

For example:

```
print("Starting data load")
```

and:

```
print("Filename:", filename)
```

These messages can help you understand what your program was doing before the problem appeared.

11. Beginner-Friendly Pseudocode

```
START

CREATE a function called load_data
    RECEIVE filename

    PRINT "Trying to load data"

    TRY
        OPEN the file
        READ the data
        CONVERT JSON into Python data
        PRINT success message
```

```
USE or return the data

EXCEPT a loading problem
    PRINT a friendly error message

CALL the function with a filename

END
```

The key structure is:

```
function
    try
        risky data operation
    except
        graceful error message
```

12. Suggested Solving Approach: Functional Approach

Use a small function such as:

```
def load_data(filename):
    ...
```

Keeping the loading code inside a function gives the program a clear responsibility.

Think of the function as a small tool:

```
filename
  ↓
load_data()
  ↓
loaded data
```

or, if something fails:

```
filename
  ↓
load_data()
  ↓
friendly error message
```

Try to keep your first version small.

Don't mix API handling, multiple files, many exceptions, and complex validation into the same exercise.

13. Easy Edge Cases

File missing

Suppose your program tries:

```
customer.json
```

but the actual file is:

```
customers.json
```

The file operation can fail even though your Python syntax is correct.

A graceful response could be:

```
Could not load the file.
```

Malformed input

Valid JSON might look like:

```
{  
    "name": "Riya"  
}
```

But this is malformed:

```
{  
    "name": "Riya"
```

The closing } is missing.

Python may successfully find the file but fail while converting its contents into JSON data.

This shows that:

```
File exists
```

does not necessarily mean:

```
File contains valid data
```

Unavailable data

With an API, the file may be replaced by an external server.

Conceptually:

```
request API
  ↓
server unavailable
  ↓
request fails
```

Instead of assuming the request always succeeds, your program should eventually be able to respond with something like:

```
Data is currently unavailable.
```

For today's exercise, you only need to practice the basic idea rather than advanced network error handling.

14. Expected Successful and Failed Output

Suppose the data is available.

Successful case

```
Trying to load data...
Data loaded successfully.
Customer: Riya
```

The basic flow is:

```
Start
  ↓
Call load_data()
  ↓
Try loading file
  ↓
Success
  ↓
Use data
```

Failed case

If the file cannot be loaded:

```
Trying to load data...
Could not load the data.
```

The flow becomes:

```
Start
↓
Call load_data()
↓
Try loading file
↓
Problem occurs
↓
except runs
↓
Friendly message
```

The important result is that the program handles the problem **gracefully instead of unexpectedly crashing**.

15. Hint Only

Start with this general shape:

```
def load_data(filename):

    try:
        # open the file
        # load the JSON data
        # print a success message

    except:
        # print a simple failure message
```

Add one temporary debugging line such as:

```
print("Trying:", filename)
```

Then test your program twice:

```
Test 1 → use a real file

Test 2 → use a filename that does not exist
```

Ask yourself:

```
Did the successful case work?

Did the failed case show a friendly message instead of crashing?
```

That is the main debugging skill to practice on **Day 69**.